

PREFACE FOR FACULTY

In real-time DSP, input sequences $x[n]$ (speech, audio, radar returns) are effectively infinite in length, while FIR filter impulse responses $h[n]$ have finite length M . Computing the entire convolution at once is impossible due to memory limits and unacceptable latency. The **Overlap-Add (OLA)** method segments the input into non-overlapping blocks of length L and performs block convolution via FFT/IFFT.

Pedagogical Strategy:

1. Formulate input signal decomposition into disjoint blocks: $x[n] = \sum_{m=0}^{\infty} x_m[n - mL]$.
 2. Establish block zero-padding requirement: $N \geq L + M - 1$.
 3. Compute frequency-domain circular convolution for each block:
$$y_m[n] = \text{IDFT}_N \text{DFT}_N x_m \cdot \text{DFT}_N h.$$
 4. Synthesize the total output by adding the overlapping $M - 1$ tails between consecutive blocks.
 5. Derive the optimal block length L_{opt} that minimizes total computational cost per output sample.
-

1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

1. **Partition** long streaming sequences into blocks suitable for Overlap-Add filtering.
 2. **Execute** complete numerical block filtering using the Overlap-Add method.
 3. **Determine** the required FFT size N to prevent aliasing within blocks.
 4. **Calculate** overall throughput and latency for real-time DSP implementations.
-

2. MATHEMATICAL FOUNDATIONS

2.1 Overlap-Add Block Decomposition

Let $x[n]$ be an infinite input sequence and $h[n]$ an FIR filter of length M . Segment $x[n]$ into non-overlapping blocks of length L :

$$x_m[n] = \begin{cases} x[n + mL], & 0 \leq n \leq L - 1 \\ 0, & \text{otherwise} \end{cases}$$

Total input: $x[n] = \sum_{m=0}^{\infty} x_m[n - mL]$.

2.2 Linear Convolution of Sub-Blocks

By Linearity of LTI systems:

$$y[n] = x[n] * h[n] = \left(\sum_{m=0}^{\infty} x_m[n - mL] \right) * h[n] = \sum_{m=0}^{\infty} (x_m[n - mL] * h[n]) = \sum_{m=0}^{\infty} y_m[n - mL]$$

Where each block convolution $y_m[n] = x_m[n] * h[n]$ has length:

$$N = L + M - 1$$

2.3 FFT Implementation & Overlap-Addition

1. Choose FFT size $N = 2^k \geq L + M - 1$.
2. Zero-pad $x_m[n]$ with $N - L$ zeros to length N .
3. Zero-pad $h[n]$ with $N - M$ zeros to length N .
4. Precompute $H[k] = \text{DFT}_N h[n]$.
5. For each block m :
 - $X_m[k] = \text{DFT}_N x_m[n]$
 - $Y_m[k] = X_m[k] \cdot H[k]$
 - $y_m[n] = \text{IDFT}_N Y_m[k]$
6. Reconstruct output $y[n]$: Overlap the last $M - 1$ points of block m with the first $M - 1$ points of block $m + 1$ and add.

3. WORKED NUMERICAL EXAMPLES

Example 13.1: Complete Numerical Overlap-Add Filtering

Problem: Filter input $x[n] = \underset{\uparrow}{1}, 2, -1, 2, 3, -2, 0, 1, 2, 1$ using FIR filter $h[n] = \underset{\uparrow}{1}, 2, 1$ ($M = 3$) using Overlap-Add with block length $L = 4$.

Solution:

- $L = 4, ; M = 3 \implies N = L + M - 1 = 4 + 3 - 1 = 6$. (Pad to $N = 6$).
- Partition input into blocks of $L = 4$:

- $x_0[n] = 1, 2, -1, 2, 0, 0$ (padded with 2 zeros)
- $x_1[n] = 3, -2, 0, 1, 0, 0$
- $x_2[n] = 2, 1, 0, 0, 0, 0$
- Filter padded: $h[n] = 1, 2, 1, 0, 0, 0$.
- **Block Convolutions** ($y_m[n] = x_m * h$):
 - $y_0[n] = 1, 2, -1, 2 * 1, 2, 1 = 1, 4, 4, 1, 3, 2$
 - $y_1[n] = 3, -2, 0, 1 * 1, 2, 1 = 3, 4, -1, 0, 2, 1$
 - $y_2[n] = 2, 1, 0, 0 * 1, 2, 1 = 2, 5, 4, 1, 0, 0$
- **Overlap-Addition** ($L = 4$ shift):

Misplaced \hline

Result:

$$y[n] = 1, 4, 4, 1, 6, 6, -1, 0, 4, 6, 4, 1$$

4. UNIVERSITY EXAMINATION QUESTIONS & MARKING RUBRIC

Question 1 (15 Marks)

(a) Describe the Overlap-Add method for linear filtering of long data sequences. Explain why zero-padding is necessary. (7 Marks) (b) Use the Overlap-Add method to filter

$x[n] = 1, 2, 0, -1, 3, 1, 2, -1$ with $h[n] = 1, 1, 1$ using block length $L = 4$. (8 Marks)

Model Answer & Step-by-Step Marking Rubric:

- **Part (a):**
 - Mathematical formulation of block decomposition and overlap additions (4 Marks)
 - Explanation of $N \geq L + M - 1$ to prevent circular aliasing (3 Marks)
- **Part (b):**
 - Block setup: $x_0 = 1, 2, 0, -1$; $x_1 = 3, 1, 2, -1$ (2 Marks)
 - Convolutions: $y_0 = 1, 2, 0, -1 * 1, 1, 1 = 1, 3, 3, 1, -1, -1$
 $y_1 = 3, 1, 2, -1 * 1, 1, 1 = 3, 4, 6, 2, 1, -1$ (4 Marks)
 - Assembly: $y[n] = 1, 3, 3, 1, (-1 + 3), (-1 + 4), 6, 2, 1, -1 = 1, 3, 3, 1, 2, 3, 6, 2, 1, -1$ (2 Marks)

5. PYTHON VERIFICATION SCRIPT

```
import numpy as np

x = np.array([1, 2, 0, -1, 3, 1, 2, -1])
h = np.array([1, 1, 1])
y = np.convolve(x, h)
print("Direct Linear Convolution:", y)
```